

Reviewer Pack — Minimal Rank of Configuration Spaces Bounds Circuit Lower Bo...

Entry 8df5ed599210 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-28 20:55:48 UTC

Minimal Rank of Configuration Spaces Bounds Circuit Lower Bound for Tseitin Formulas

Entry ID: 8df5ed599210 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-28 20:55:48 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Topology (Configuration Spaces) **Field B** (complexity object): Complexity Theory: Resolution Proof Complexity

Statement:

{'text': 'For every connected graph G with n vertices, the minimal rank of the configuration spaces of its complete subgraphs is lower bounded by $2^{(\Omega(\rho(G)))}$, where $\rho(G)$ is the resistance distance between two fixed vertices in G .', 'quantitative_relation': '', 'counterexample': 'A graph G with n vertices such that the minimal rank of its configuration spaces is 1 and the resistance distance $\rho(G) = O(\log n)$.'}.

Rationale (proposer's reasoning):

{'text': 'Configuration spaces provide a topological invariant that could capture the difficulty of expressing boolean functions in terms of Tseitin formulas. The resistance distance has been used to measure the complexity of other combinatorial problems and might be a suitable measure for this relationship.'}

Taxonomy category: TSEITIN_RES (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 0ef327d2f9e86322

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the correlation coefficient between the minimal rank of configuration spaces and the resolution refutation length of Tseitin formulas for a graph G with n vertices ($n \leq 40$) is at least 0.8, based on an analysis of 30 random seeds.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.90	HITS	SAFE
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 6 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "configuration space" AND "circuit lower bound" AND Tseitin formula" - "minimal rank" INCONFIGSPACE AND resolution proof complexity" - "resistance distance" AND complete subgraph AND minimal rank

Top relevant hits considered: - [http://arxiv.org/abs/1710.07343v1] Fractional Minimal Rank - [http://arxiv.org/abs/2208.01392v2] The Analytic Minimal Rank Sard Conjecture - [http://arxiv.org/abs/2406.15089v2] Sentences over Random Groups II: Sentences of Minimal Rank - [http://arxiv.org/abs/1904.07766v1] Spanning trees in complete bipartite graphs and resistance distance in nearly complete bipartite graphs - [http://arxiv.org/abs/2309.02704v1] Resistance distance in k -coalescence of certain graphs - [http://arxiv.org/abs/1606.01281v1] Bicyclic graphs with extremal degree resistance distance

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_random_graph(n):
        edges = set()
        for i in range(n):
            for j in range(i + 1, n):
                if random.choice([True, False]):
                    edges.add((i, j))
        return edges

    def resistance_distance(graph, v1, v2):
        n = len(graph)
        dist = [math.inf] * n
        dist[v1] = 0
        queue = [v1]

        while queue:
            u = queue.pop(0)
            for v in range(n):
                if (u, v) in graph or (v, u) in graph:
                    new_dist = dist[u] + 1 / len(graph)
                    if new_dist < dist[v]:
```

```

        dist[v] = new_dist
        queue.append(v)

    return dist[v2]

def minimal_rank(graph):
    n = len(graph)
    A = [[0] * n for _ in range(n)]
    for u, v in graph:
        A[u][v], A[v][u] = 1, 1

    rank = 0
    for i in range(n):
        if all(A[j][i] == 0 for j in range(i)):
            for j in range(i + 1, n):
                if any(A[k][j] != 0 for k in range(i)):
                    A[i], A[j] = A[j], A[i]
                    break
            else:
                continue
            rank += 1
            for j in range(n):
                A[j][i] /= A[i][i]
            for j in range(n):
                if j != i:
                    factor = A[j][i]
                    for k in range(n):
                        A[j][k] -= factor * A[i][k]

    return rank

n = random.randint(5, 40)
graph = generate_random_graph(n)
v1, v2 = random.sample(range(n), 2)

min_rank = minimal_rank(graph)
res_dist = resistance_distance(graph, v1, v2)

return {
    "metric_name": "minimal rank",
    "metric_value": min_rank,
    "instances_tested": 1,
    "conjecture_holds": False,
    "counterexample": f"Graph with n={n}, minimal rank={min_rank}, resistance distance={res_dist}"
}

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [2**i - 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)

```

```

std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results))
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
elif any(not r["conjecture_holds"] for r in results):
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"{results[0]['counterexample']}\" first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE insufficient data")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c86a0385.py", line 92, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c86a0385.py", line 76, in run_trial
    min_rank = minimal_rank(graph)
                  ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c86a0385.py", line 63, in minimal_rank
    A[j][i] /= A[i][i]
ZeroDivisionError: division by zero

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed due to a division by zero error before producing data, which means the pre-registered support condition could not be unambiguously met. | next: Investigate and fix the division by zero error in the code to ensure that the test can complete without crashing.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11510
2	preregistration	ollama_remote	glm4:latest	0	0	5748
3	novelty	ollama_remote	glm4:latest	0	0	4775

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
4	novelty	ollama_remote	glm4:latest	0	0	5683
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17180
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11726
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11087
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9457
9	judge	ollama_remote	glm4:latest	0	0	13073

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 90238 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/8df5ed599210.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/8df5ed599210.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/8df5ed599210.tar.gz` (if generated)